

ESSELMOB — Knowledge Transfer Guide

For Junior Developers Joining the Mobile App Team

Essel Projects — Mobile Engineering

July 2026

Contents

1. Purpose of This Document	2
2. What Does This App Actually Do?	2
3. Environment Setup	2
3.1 Prerequisites	2
3.2 Get the code running	3
3.3 Where the app talks to	3
4. The Mental Model	3
5. Folder-by-Folder Walkthrough	4
6. Worked Example: Tracing the Login Flow	6
7. Worked Example: CC Budget Amendment Verification (List → Detail → Approve)	6
7.1 The list screen	7
7.2 The detail screen	7
7.3 Why buttons are “dynamic”	7
7.4 What’s the same across all three verification modules	7
8. Core Concepts Glossary	8
9. How to Add a Small Feature (Checklist)	9
10. Conventions, Quirks, and Things to Know Before You Touch Code	10
11. Debugging Tips	10
12. Build & Release, in Brief	11
13. Quick Reference: Where Do I Find...?	11
14. Who to Ask / Where to Go Next	12

1. Purpose of This Document

This is a Knowledge Transfer (KT) guide for junior developers who are new to **ESSELMOB**, the React Native mobile app for Essel Projects' ERP system. It assumes you know basic JavaScript/TypeScript and have heard of React, but have not worked in this codebase before.

By the end of this document you should be able to:

- Set up the project and run it on your machine
- Explain, in your own words, how data flows from a button tap to the server and back
- Find your way around the folder structure without getting lost
- Add a small new screen or field by following existing patterns
- Know where to look when something breaks

Read this alongside `MOBILE_APP_DOCUMENTATION.md` in the project root, which is the detailed technical reference (full endpoint list, every slice, every component). This KT guide is the “learn by walking through it” companion to that reference.

2. What Does This App Actually Do?

ESSELMOB is a mobile companion to Essel Projects' web ERP. Field staff and approvers use it to:

1. **Log in** with an employee ID, then choose to enter either as a regular **employee** or as a **role/approver** (a second password unlocks role-based menus).
2. **See a notification inbox** of things waiting for their action, with a badge count on the app icon (like unread emails).
3. **Verify / approve** three kinds of ERP transactions from their phone:
 - **CC Budget Amendments** — cost-center budget change requests
 - **DCA Budget Amendments** — a second type of budget amendment
 - **Indents** — purchase requisitions

Each of these three verification flows works the same way: a **list screen** shows pending items, tapping one opens a **detail screen** with line items, remarks history, and Approve/Reject/Return buttons.

If you understand *one* of these three flows deeply, you understand all three — they're intentionally built on the same pattern. We'll trace CC Budget Amendment as the worked example in Section 6.

3. Environment Setup

3.1 Prerequisites

- Node.js (LTS) and npm

- Git
- Android Studio (for the Android emulator) — Android is the primary target platform
- Xcode, if you're on macOS and testing iOS (secondary platform)
- The Expo Go app on your phone, OR a development build, for testing on a real device

3.2 Get the code running

```
git clone https://github.com/ANAYAK06/esselAppMobile.git
cd esselAppMobile
npm install
npm start          # starts the Expo dev server
```

From the Expo dev server output you can:

- Press a to open on a connected Android emulator/device
- Press i to open on an iOS simulator (macOS only)
- Scan the QR code with Expo Go on a physical phone

Other useful scripts:

```
npm run android    # expo run:android – full native build
npm run ios        # expo run:ios
npm run lint       # ESLint check
```

There are currently **no automated tests** in this project — don't go looking for a test suite, there isn't one yet. Manual testing on device/emulator is how changes get verified today.

3.3 Where the app talks to

The whole app points at one backend, configured in `src/service/apiConfig.ts`:

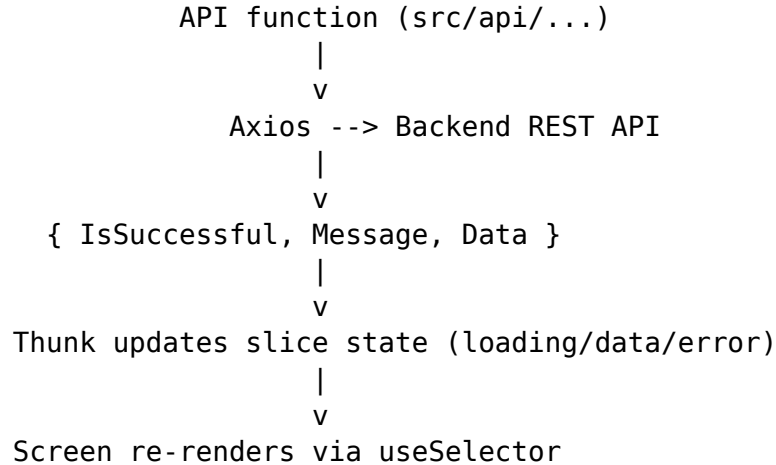
`https://myesselapi.esselprojects.com/api`

Note: this is the **production** API, and it's used even in local development (`__DEV__` mode) — there's no separate dev/staging backend wired up right now. Be careful: actions you take while developing (like approving a budget amendment) hit the real production system.

4. The Mental Model

Before touching code, hold this picture in your head:

```
Screen (app/...) --dispatches--> Redux Thunk (src/slice/...)
                                |
                                v
```



Three things happen on almost every screen in this app:

1. A **screen component** (in app/) reads state with useSelector / useDispatch and dispatches an action with useDispatch / dispatch.
2. A **slice** (in src/slice/) owns that piece of state and contains createAsyncThunk functions that call the API and update loading/data/error state as the request progresses.
3. An **API module** (in src/api/) is a thin wrapper around axios that knows the exact backend endpoint and shapes the request/response.

Once this clicks, every new feature is “add a screen, add a slice (or extend one), add an API function” — the same three steps, repeated.

5. Folder-by-Folder Walkthrough

ESSELMOB/	
├─ app/	screens – Expo Router file-based routing
├─ src/	
│ ├─ api/	axios calls, one folder per domain
│ ├─ slice/	Redux Toolkit state, one folder per domain
│ ├─ store/	Redux store setup + typed hooks
│ ├─ components/	reusable UI pieces
│ ├─ service/	apiConfig.ts (base URL), s3Config.ts (file/document URLs)
│ ├─ hooks/	custom hooks (e.g. app badge)
│ └─ utils/	small standalone helpers
├─ android/	native Android project (Gradle) – rarely touched
├─ assets/	images, icons, fonts
├─ app.json	Expo app config (name, version, plugins)
└─ eas.json	EAS Build profiles (dev / preview / production)

Rule of thumb: app/ is *where a screen lives on the navigation stack*. src/ is *everything that screen uses* — the actual logic. If you’re asked “where does the CC Budget list screen live,” the answer is app/(inbox)/verification/cc-budget/list.tsx, but

the *data fetching* for it lives in `src/slice/budget/ccBudgetAmendmentSlice.ts` and `src/api/budget/ccBudgetAmendmentAPI.ts`.

app/ — routing groups

Folders in parentheses, like `(auth)`, are **route groups**: they organize files but don't appear in the URL/path.

Group	Contains
<code>(auth)</code>	login, login-options, biometric-setup (placeholder)
<code>(dashboard)</code>	employee-dashboard (placeholder), role-dashboard (fully built)
<code>(inbox)</code>	inbox screen + verification list/detail screens for cc-budget, dca-budget, indent

Each folder can have its own `_layout.tsx`, which wraps its child screens in a navigation container (usually a `Stack` with `headerShown: false`, since headers are custom-built with `AppHeader`).

src/api/ and src/slice/ — mirrored structure

These two folders mirror each other on purpose:

```
src/api/budget/ccBudgetAmendmentAPI.ts <--> src/slice/budget/ccBudgetAmendmentSlice.ts
src/api/indent/indentAPI.ts             <--> src/slice/indent/indentSlice.ts
src/api/auth/authAPI.ts                 <--> src/slice/auth/authSlice.ts
```

If you're looking for how a piece of data is fetched, find its slice first (it tells you *what* is fetched and *when*), then follow the import to the matching API file (which tells you *which endpoint* and *what shape*).

src/components/ — organized by feature, not by type

Unlike some codebases that dump all components in one flat folder, this one groups by feature area: `dashboard/`, `inbox/`, `verification/`, `common/`. The most important folder for you as a junior is `verification/`, because of two **generic, reusable** components:

- `GenericVerificationList` — a reusable list screen shell
- `GenericVerificationDetail` — a reusable detail screen shell

These exist so that CC Budget, DCA Budget, and Indent don't each reinvent list/detail UI from scratch. New verification-style features should reuse these where possible instead of copy-pasting a whole screen.

6. Worked Example: Tracing the Login Flow

This is the best first flow to read end-to-end because it's short and touches every layer.

Step 1 — user opens the app. `app/index.tsx` is the splash screen. It runs a short animation, then calls `router.replace('/login')`.

Step 2 — employee ID + password. `app/(auth)/login.tsx` uses **Formik + Yup** for the form (employeeId, password, both required, password min 6 chars). On submit, it dispatches:

```
dispatch(validateEmployee({ employeeId, password })))
```

Step 3 — the thunk. In `src/slice/auth/authSlice.ts`, `validateEmployee` is a `createAsyncThunk`. It calls `validateEmployeeAPI(credentials)` from `src/api/auth/authAPI.ts`, which POSTs to:

```
POST /Security/GetValidEmployee { Username, Password }
```

Step 4 — response handling. The API returns `{ IsSuccessful, Message, Data: { isValidUser } }`. Back in the slice's `extraReducers`, `validateEmployee.fulfilled` sets `isAuthenticated = true`, `employeeValidated = true`, and saves `employeeId` to `AsyncStorage` (so a future app restart can restore the session).

Step 5 — screen reacts. Back in `login.tsx`, a `useEffect` watches `success.validateEmployee` from Redux state. When it flips to true, the screen navigates to `/login-options`.

Step 6 — choose access type. `app/(auth)/login-options.tsx` offers two paths:

- **Employee Portal** → dispatches `getEmployeeDetails(employeeId)` → `GET /Accounts/GetEmployeeDetailsbyUser` → routes to `/employee-dashboard` (currently a placeholder screen, not yet built out).
- **Role Portal** → requires a second “role password” → dispatches `validateUser` (`POST /Security/GetValidUser`) to get a `roleId` and profile, then `getMenu(roleId)` (`GET /Accounts/GetMenu`) → routes to `/role-dashboard` (fully implemented).

Step 7 — session persistence. `src/store/store.ts` configures `redux-persist` to persist only the auth slice to `AsyncStorage`. On a fresh app launch, `loadFromStorage()` (dispatched from `login.tsx`) rehydrates the session, so a returning user with a valid cached session skips straight to their dashboard instead of re-entering credentials.

That's the whole flow: **form → thunk → axios → backend → state update → navigation**. Every other feature in the app follows this same shape.

7. Worked Example: CC Budget Amendment Verification (List → Detail → Approve)

This is the flow to study once you've understood login, because it's the template for **all three verification modules**.

7.1 The list screen

`app/(inbox)/verification/cc-budget/list.tsx:`

1. Reads `uid` and `roleId` out of the auth slice.
2. On mount (and whenever the screen regains focus, via `useFocusEffect`), dispatches `fetchAmendmentsList({ roleId, uid })`.
3. That thunk (in `src/slice/budget/ccBudgetAmendmentSlice.ts`) calls `getApprovalCCAmendBudgetDetails` in the API file, hitting:
`GET /Accounts/GetApprovalCCAmendBudgetCDetails?Roleid=...&UID=...`
4. Results are stored in `ccBudgetAmendment.amendments`, read via the `selectAmendments` selector, and rendered as a list of `VerificationItemCard` components.
5. Tapping a card navigates to `/verification/cc-budget/[amendId]`.

7.2 The detail screen

`app/(inbox)/verification/cc-budget/[id].tsx:`

1. Reads the `id` (amendment ID) from the route params via `useLocalSearchParams`.
2. Dispatches `fetchAmendmentById` → `GET /Accounts/GetApprovalCCAmendBudgetById?AmendId=...` to load the full detail (old vs. new budget figures, shown with `TrendingUp/TrendingDown` icons).
3. Also checks whether supporting documents exist (`checkDocumentsExist` → `Getcuploadocsexists`), and if so builds a preview/download link using `buildCCBudgetAmendmentUrl()` from `src/service/s3Config.ts` — this turns a filename the backend gave us into a full S3 URL.
4. Shows the dynamic action buttons (see 7.3).
5. On `Approve/Reject`, dispatches `approveAmendment` with an entered remarks string → `PUT /Accounts/ApproveCostCenterBudgetAmend...`

7.3 Why buttons are “dynamic”

You’ll notice the action buttons aren’t hardcoded as `<Button>Approve</Button>` `<Button>Reject</Button>`. Instead, `DynamicActionButtons` (`src/components/verification/Actions/DynamicActionButtons.tsx`) calls:

`GET /Accounts/GetStatuslist?MOID=...&ROID=...&ChkAmt=...`

...and renders whatever action set the backend says is valid for this module/role/amount combination. This is a deliberate design choice: **the backend controls the workflow rules**, and the app just renders whatever it’s told. If you’re asked “why isn’t there a Return button for this amendment,” the answer is in the backend’s `GetStatuslist` response, not in the mobile app’s code.

7.4 What’s the same across all three verification modules

Concept	CC Budget	DCA Budget	Indent
List thunk	fetchAmendmentsList	fetchDCAAmendmentsList	fetchVerificationGrid
Detail thunk	fetchAmendmentById	fetchDCAAmendmentById	fetchIndentFullDetails
Action thunk	approveAmendment	approveDCAAmendment / updateApprovalD-CAAmendment	verifyIndent
Dynamic actions	DynamicActionButtons (shared component)	same	same
Remarks history	shared Remark-sHistorySection	shared	IndentRemarksHistory (indent-specific variant)

If you're building a **fourth** verification module in the future, copy this table's pattern: one list thunk, one detail thunk, one action thunk, reuse `DynamicActionButtons` and the remarks components.

8. Core Concepts Glossary

If any of these are new to you, look them up before diving deeper — the whole codebase assumes familiarity with them.

Expo Router — file-based routing for React Native, similar in spirit to Next.js. A file at `app/(inbox)/inbox.tsx` automatically becomes the route `/inbox`. Folders in parentheses group routes without adding a URL segment. `[id].tsx` is a dynamic route segment.

Redux Toolkit "slice" — a bundle of state + reducers + action creators for one feature area, created with `createSlice`. Each domain (auth, indent, budgets, etc.) has its own slice.

createAsyncThunk — a Redux Toolkit helper for async logic (typically an API call). It automatically dispatches pending / fulfilled / rejected actions, which slices handle in `extraReducers` to update loading / data / error state.

redux-persist — saves part of the Redux store to `AsyncStorage` so it survives an app restart. Only the auth slice is persisted here.

NativeWind — lets you use Tailwind CSS utility classes (`className="flex-1 bg-white"`) in React Native components. You'll see it mixed with plain `style={{ ... }}` objects in this codebase, especially for gradients and dynamic/conditional styling.

Formik + Yup — form state management (Formik) and schema-based validation (Yup). Used on the login and role-password forms.

Axios — the HTTP client used for every API call. There's no shared/centralized Axios instance in this project — each API file constructs requests individually using the shared `API_BASE_URL` constant.

AsyncStorage — React Native's persistent key-value storage (similar to `localStorage` on the web). Used both directly (auth slice caches `employeeId`, `userData`, etc.) and indirectly via `redux-persist`.

EAS Build / EAS Update — Expo's cloud build and OTA (over-the-air) update service. `eas.json` defines build profiles (development, preview, production). Because `expo-updates` is configured, most JS-only changes can ship as an OTA update without a full app store release.

9. How to Add a Small Feature (Checklist)

Say you're asked to add a new read-only screen that shows, e.g., a list of "My Leave Requests." Here's the pattern to follow, based on how everything else in this app is built:

1. **API function** — add a new file (or extend an existing one) under `src/api/<domain>/`, following the existing style: an async function, `axios.get/post/put`, typed request/response interfaces, `try/catch` that rethrows `error.response.data` when present.
2. **Slice** — add a `createAsyncThunk` in a slice under `src/slice/<domain>/` that calls your new API function, plus `loading / error / data` fields in the slice state, handled in `extraReducers`. Export a `select*` selector for the data.
3. **Register the reducer** — add your slice's reducer to `rootReducer` in `src/store/store.ts` (and decide whether it belongs in the whitelist for persistence — most slices should NOT be persisted).
4. **Screen** — add a new file under `app/`, in the right route group. Use `useAppDispatch / useAppSelector` (from `src/store/hooks.ts`), dispatch your thunk in a `useEffect`, and render `loading/error/data` states. Reuse `AppHeader` for the top bar.
5. **Wire up navigation** — if it's reached from a menu or another screen, add a `router.push('/your-route')` call there.
6. **Test on device/emulator** — there's no automated test suite, so manual verification is the only safety net right now. Check both the happy path and what happens when the API call fails.

If your feature is another "list of pending things + detail + approve/reject," strongly consider reusing `GenericVerificationList / GenericVerificationDetail / DynamicActionButtons` instead of building a new screen from scratch (see Section 7).

10. Conventions, Quirks, and Things to Know Before You Touch Code

- **Every API response follows { IsSuccessful, Message, Data }.** Some slices also defensively check for a Success flag or just check whether Data is non-empty, because not every backend endpoint is 100% consistent about which flag it sets. When in doubt, log the raw response first.
 - **The `__DEV__` branch of `apiConfig.ts` points at production,** not a local/staging server. There is currently no safe sandbox backend — be careful with write actions (approve/reject/save) while developing.
 - **Two screens are placeholders, not real implementations:** `(dashboard)/employee-dashboard.tsx` and `(auth)/biometric-setup.tsx` both just render a “Dummy Screen Loaded” view. Don’t assume they work — if you’re asked to build out the employee dashboard, you’re building it from scratch, not fixing a bug.
 - **`(dashboard)/_layout.tsx` has a copy-paste artifact** — its function is still named `AuthLayout` (leftover from being copied from the auth layout file). It functions correctly (renders a headerless Stack), but the naming is misleading if you go looking for it.
 - **Console logging is heavy and intentional** — you’ll see lots of `console.log(...)` calls prefixed with small emoji markers (a magnifying glass for an outgoing request, a check mark for success, a cross or siren for errors), especially in auth and API code. This is the primary debugging tool in this codebase (see Section 11). Keep this pattern when you add new async logic — it’s how the team traces issues without a proper logging/monitoring setup.
 - **S3 file URLs are never stored as full URLs** — the backend gives back a relative file path, and the app builds the full S3 URL client-side via helpers in `src/service/s3Config.ts` (e.g. `buildCCBudgetAmendmentUrl`). If you add a new document-upload feature, add a new folder constant + helper function there rather than hardcoding a URL.
 - **Styling mixes `NativeWind` `className` and inline style objects** in the same component — this is normal in this codebase, not a mistake. `className` handles layout/spacing/typography; `style` is used for things `NativeWind` can’t easily express (gradients, shadows, computed/conditional colors).
 - **No automated tests exist.** Don’t be surprised there’s no `__tests__` folder — manual testing on a device/emulator is the current process.
-

11. Debugging Tips

1. **Read the console logs.** Nearly every thunk and API call logs its request payload, the raw response, and any error — search for the emoji markers (magnifying glass = outgoing request, check mark = success, cross/siren = error) to quickly scan npm start’s terminal output or the in-app dev console.
2. **NetworkDebug component** (`src/components/debug/NetworkDebug.tsx`), shown on the login screen, is a dev-only helper for diagnosing connectivity issues to the API.

3. **Redux DevTools** are enabled in development (`devTools: __DEV__` in `store.ts`) — use the React Native Debugger or Expo’s dev tools to inspect state and actions live.
 4. **Check AsyncStorage state** if login/session behavior looks wrong — remember only the auth slice persists across restarts; everything else resets.
 5. **If a screen shows no data**, check three things in order: (1) is the thunk actually being dispatched (add a log), (2) what does the raw API response look like (log it in the API function), (3) does the response match the `IsSuccessful/Data` shape the slice expects.
 6. **If action buttons don’t show up** on a verification detail screen, check the `GetStatuslist` response for that `MOID/ROID/ChkAmt` combination — remember, the button set is driven entirely by the backend (Section 7.3).
-

12. Build & Release, in Brief

- Builds are produced via **EAS Build** (`eas.json`): development (internal, dev client), preview (internal APK), production (auto-incrementing version).
 - Most JS-only changes should ship as an **OTA update** via expo-updates rather than a full rebuild — this is called out explicitly in the project’s `CLAUDE.md` guidance. Native module changes (new native dependency, permission change, etc.) still require a full rebuild/release.
 - Android (`com.anayak06.ESELMOB`) is the primary target; iOS is secondary and gets less testing attention.
-

13. Quick Reference: Where Do I Find...?

I want to know...	Look here
What screens exist and how they’re routed	<code>app/</code> folder structure, Section 5
What API endpoints exist	<code>src/api/**</code> or <code>MOBILE_APP_DOCUMENTATION.md §6</code>
What state a feature holds	matching file in <code>src/slice/**</code>
How a form validates input	<code>useFormik</code> + <code>Yup</code> schema at the top of a screen file
How file/document links are built	<code>src/service/s3Config.ts</code>
What the base API URL is	<code>src/service/apiConfig.ts</code>
Shared UI I should reuse	<code>src/components/common/</code> , <code>src/components/verification/Generic*</code>
Build/version config	<code>app.json</code> , <code>eas.json</code>

14. Who to Ask / Where to Go Next

- For deep technical reference (every endpoint, every slice, full component inventory), read `MOBILE_APP_DOCUMENTATION.md` in the project root.
- For backend endpoint behavior and response contracts beyond what's visible from the mobile app, check the backend/API documentation and, if needed, confirm directly with the backend team — several endpoints (like `GetStatuslist`) drive UI behavior in non-obvious ways.
- When you're unsure whether something is "broken" or "just not built yet," check Section 10 of this document first (placeholder screens, known quirks) before assuming it's a bug you introduced.

Welcome to the team — the fas